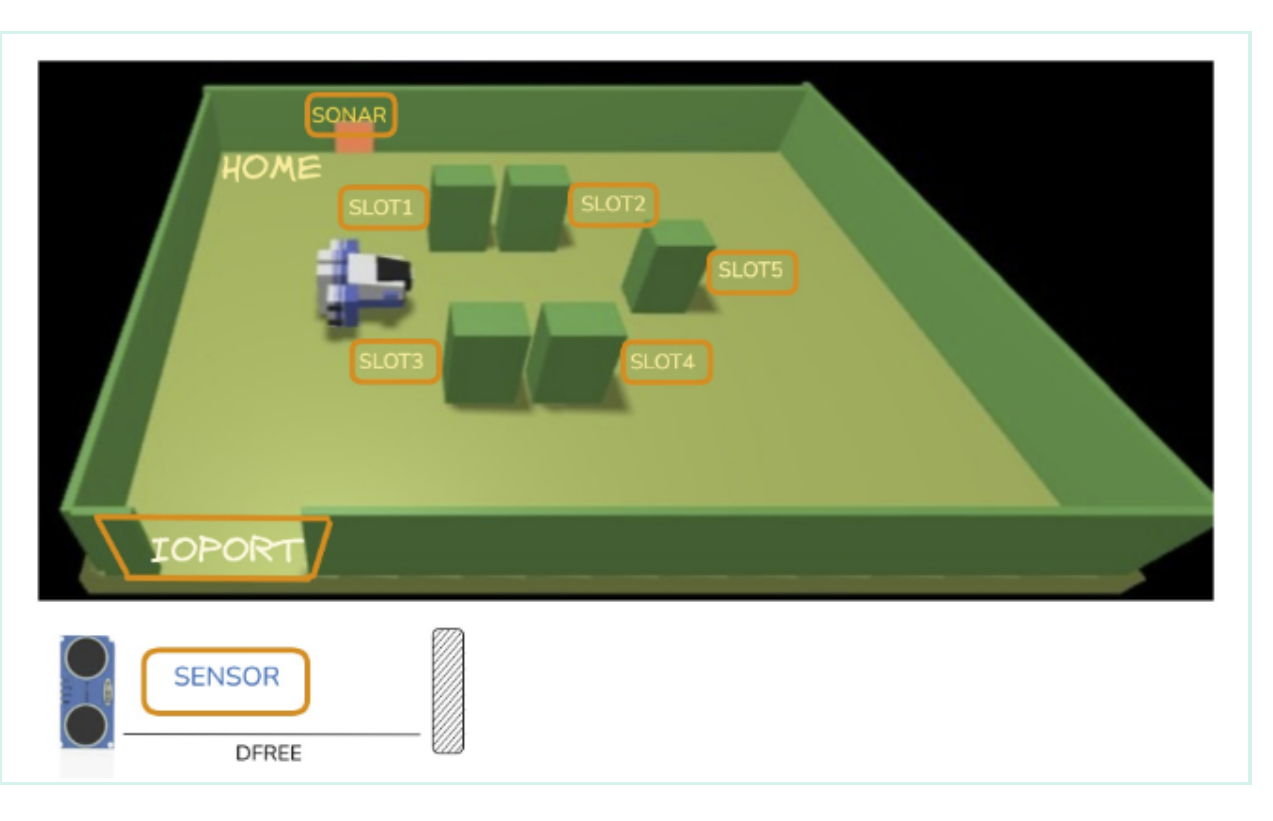
TF26 - Sprint2

Goal Sprint2: Progettare e realizzare l'attore `cargorobot` e integrarlo con l'ambiente virtuale `RobotSmart26` per eseguire la missione autonoma di trasferimento container (IOPort -> slot 5 -> slot assegnato -> HOME). Modellare il componente `marker` per la sincronizzazione del processo di marcatura nello slot 5.

Introduction

A *Maritime Cargo shipping company* (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**).

The *hold* is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides `4 slots` to store the containers and a slot named **slot5**.



In the picture above:

- The **slots1-4** depict the hold areas reserved to store one *container* each,
- The **slot5** depicts an area, where the *cargorobot* must temporarily store a container, before to place it in one of the *slots1-4*. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.
- The **IOPort** is a device with a **pushbutton** and **a display**. The *pushbutton* is pressed by the customer in order to to send a request to load a container on the cargo. The *display* is used to show the answer to the request and to show the current state of the *hold*.
- The **sensor** associated to the *IOPort* is a device (a *sonar*) used to detect the presence of a container, when it measures a distance D, such that `D < DFREE/2`, during a reasonable time (e.g. 3 secs).

Requirements

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer `retrylater`, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the `slots1-4` are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request* to load is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message **'Service working'**, when it is all is going well
- the message **'Out of service'** if the *sonar sensor* measures a distance `D > DFREE` for at least 3 secs (perhaps a failure of the *sonar*).

Requirement analysis

L'analisi dei requisiti è stata svolta nello Sprint0.

Problem analysis

Il ruolo di cargorobot come componente intermedio

Come evidenziato nell'analisi dei requisiti dello Sprint 0, RobotSmart26 è il servizio esterno individuato per governare il robot a guida differenziale (DDR), in quanto riduce l'abstraction gap calcolando autonomamente il path-planning a partire da coordinate cartesiane. Tuttavia, RobotSmart26 è un componente infrastrutturale agnostico rispetto al dominio applicativo: non conosce i concetti di stiva, container, slot riservato, marcatura barcode o fasi di carico.

- Separazione delle responsabilità (SRP): Far interagire direttamente cargoservice con RobotSmart26 sovraccaricherebbe l'orchestratore di dettagli operativi legati alle tratte e creerebbe un accoppiamento tecnologico stringente verso il simulatore.
- cargorobot come attore intermediario: Si introduce cargorobot come entità autonoma dotata di ciclo di vita proprio (QActor). Riceve da cargoservice l'ordine di trasporto ad alto livello (transportContainer(SLOT, X, Y)), coordina la sequenza di tappe e traduce la missione in chiamate verso il servizio robot.

Canale di Comunicazione verso RobotSmart26

Analizzando le interfacce esposte da RobotSmart26:

1. MQTT: Richiede la presenza e la manutenzione di un broker aggiuntivo (come Mosquitto), oltre a dover gestire manualmente l'impacchettamento e la correlazione asincrona dei messaggi su topic distinti (robotsmart26in e robotsmart26in_out).
2. CoAP: Orientato prevalentemente all'osservabilità delle risorse (es. stato di robotmnemo). Il suo uso per il comando richiederebbe la riscrittura a mano del layer di richiesta/risposta.
3. ExternalQActor su TCP (Scelta di Progetto): Poiché RobotSmart26 è esso stesso sviluppato mediante l'infrastruttura QAk, è possibile referenziarlo direttamente come ExternalQActor su un contesto remoto (ctxrobotsmart). L'infrastruttura QAk gestisce in modo trasparente e nativo la serializzazione dei messaggi e il socket TCP, senza intermediari o librerie esterne.

Livello di astrazione dei comandi di movimento

Il componente software RobotSmart26 non espone una singola modalità di pilotaggio, bensì diversi livelli di astrazione per il controllo della navigazione del DDR. Nell'ottica di ridurre l'abstraction gap senza introdurre complessità non richieste dal committente, è necessario analizzare comparativamente ciascuna alternativa:

1. Comandi elementari di navigazione: step e move
    - **Definizione e Semantica**: Costituiscono il livello di interazione a più basso livello esposto dal servizio. Comprendono primitive atomiche come il singolo passo in avanti calibrato su un tempo prestabilito (step(TIME)) con relativa risposta di esito (stepdone(V) / stepfailed(DURATION, CAUSE)), e comandi asincroni di rotazione o arresto inviati via dispatch (move(M) con M in {l, r, a, d, h}).
    - **Contesto d'uso ideale**: Questa modalità è concepita per problemi di esplorazione euristica o mappatura dinamica, ovvero per scenari in cui lo spazio di manovra non è noto a priori e il robot deve procedere per tentativi analizzando l'ambiente passo dopo passo (ad esempio costeggiando un perimetro ignoto).
    - **Valutazione critica per il progetto**: Nel nostro scenario, l'ambiente della stiva è rettangolare, statico e con coordinate ben definite note a priori fin dalla fase di analisi (HOME, IOPort, slot5, slot1..4). Optare per questo approccio costringerebbe l'attore cargorobot a farsi carico della gestione della bussola interna, della verifica dell'orientamento, del conteggio dei singoli passi e della ri-pianificazione manuale in caso di scostamento. Ciò reintrodurrebbe all'interno dell'applicazione una complessità algoritmica di basso livello che è già stata risolta a monte dal componente RobotSmart26, annullando di fatto il beneficio della sua adozione.
2. Pianificazione ed esecuzione disaccoppiate: buildPlan seguito da doplan
    - **Definizione e Semantica**: Questo livello separa nettamente la fase di calcolo del tragitto da quella di attuazione dei movimenti. L'invio della richiesta buildPlan(PX, PY, TX, TY) istruisce il servizio a elaborare una sequenza formale di passi e rotazioni ottimali per passare dalle coordinate di partenza (PX, PY) a quelle di destinazione (TX, TY), restituendo il piano nella reply buildPlanDone(PLAN). Successivamente, il chiamante inoltra tale piano tramite doplan(PLAN, STEPTIME) e attende l'esito finale (doplandone(ARG) oppure doplanfailed(PLANTODO)).
    - **Contesto d'uso ideale**: È la soluzione ottimale in contesti dinamici o critici, in cui il software di alto livello deve ispezionare, adattare o alterare il percorso prima che i motori vengano azionati (ad esempio per verificare che la traiettoria pianificata non attraversi aree temporaneamente interdette, per stimare costi energetici o per iniettare checkpoint intermedi).
    - **Valutazione critica per il progetto**: Nessun requisito del committente richiede la modifica o l'ispezione runtime del tragitto. Le tratte tra le postazioni della stiva sono predeterminate e non presentano ostacoli dinamici non cooperativi. L'introduzione di questa coppia di chiamate aggiungerebbe una transizione di stato intermedia e una responsabilità di gestione del payload del piano all'interno di cargorobot, senza portare alcun valore aggiunto tangibile al controllo della missione.
3. Navigazione a coordinate assolute: moverobot
    - **Definizione e Semantica**: Rappresenta il livello di astrazione più elevato fornito da RobotSmart26. Con una singola richiesta moverobot(TARGETX, TARGETY, STEPTIME), il chiamante specifica esclusivamente la cella cartesiana di arrivo sulla griglia della stiva e il tempo di calibrazione del passo. Il servizio calcola internamente l'orientamento iniziale, pianifica il percorso ed esegue i passi necessari, restituendo un'unica risposta conclusiva: moverobotdone (o moverobotok(ARG)) in caso di arrivo a destinazione, oppure moverobotfailed(PLANDONE, PLANTODO) in caso di ostacolo imprevisto lungo la rotta.
    - **Contesto d'uso ideale**: È indicato per scenari di navigazione deterministica all'interno di ambienti a griglia completamente noti in cui il chiamante necessita di delegare per intero la cinematica e il calcolo del percorso al controller del robot.
    - **Valutazione critica per il progetto**: L'approccio che minimizza l'abstraction gap. Permette a cargorobot di esprimere la missione ad alto livello secondo la semantica del dominio applicativo ("raggiungi IOPort", "raggiungi lo slot 5", "raggiungi lo slot riservato", "torna in HOME") senza dover gestire strutture dati relative al cammino. Inoltre, preserva la manutenibilità del codice applicativo confinando la logica di movimento all'interno del servizio preposto.

Primitiva / Livello	Input / Output	Vantaggi	Svantaggi rispetto al caso d'uso	Decisione
step / move	`step(TIME) → stepdone / stepfailed` `move(M)` (dispatch)	Massimo controllo a grana fine su ogni singolo movimento del robot.	Reintroduce in cargorobot l'onere del path-planning, della gestione dell'orientamento e della mappa della stanza.	**Scartata**: abstraction gap troppo elevato per una stiva statica con posizioni fisse.
buildPlan + doplan	`buildPlan(PX, PY, TX, TY) → buildPlanDone(PLAN)` `doplan(PLAN, STEPTIME) → doplandone / doplanfailed`	Permette al chiamante di ispezionare, adattare o memorizzare la sequenza di passi prima dell'esecuzione.	Introduce una macchina a stati più complessa per manipolare e validare il piano, privi di controparte nei requisiti.	**Scartata**: non è richiesta l'ispezione o la manipolazione dinamica delle traiettorie.
moverobot	`moverobot(TARGETX, TARGETY, STEPTIME) → moverobotok / moverobotfailed`	Minimo abstraction gap: delega completa del percorso a partire da coordinate cartesiane note a priori.	Non permette di intervenire sul percorso durante la pianificazione intermedia.	**Adottata**: soluzione più pulita e coerente con il principio di singola responsabilità (SRP).

Reazione ai Fallimenti di Movimento

Se RobotSmart26 risponde con moverobotfailed(PLANDONE, PLANTODO) (es. percorso ostruito o guasto simulato), cargorobot interrompe la missione, entra in uno stato di sicurezza (halted) e notifica cargoservice tramite il dispatch transportFailed. La decisione su come gestire il sistema (annullamento della prenotazione o stato di allarme) rimane in capo all'orchestratore.

Modellazione del Marker (Slot 5)

Il testo del problema descrive il marker come un dispositivo situato nello slot 5 per l'etichettatura barcode temporanea. Nell'analisi del problema emergono due opzioni:

1. Attore QActor separato (marker): Riceve la notifica di container depositato, calcola il tempo di etichettatura ed emette un dispatch di avvenuta marcatura.
2. Attesa gestita all'interno della sequenza di cargorobot: La marcatura è un'operazione atomica che blocca il robot in slot5 per un tempo prefissato prima che possa proseguire nuovamente il container verso lo slot di destinazione.

Per evitare la proliferazione di attori passivi, si valuta di modellare la permanenza in slot5 nello stato atSlot5 tramite una transizione temporizzata esplicita (whenTime 3000), che simula il completamento dell'attività di marcatura prima della ripartenza verso lo slot finale.

Gestione degli Slot e Modellazione della Stiva (Hold)

Nel corso dello Sprint 1, la classe Hold si limitava a tenere traccia della disponibilità numerica e dello stato astratto degli alloggiamenti (FREE, RESERVED, OCCUPIED) per abilitare le risposte di accettazione o rifiuto delle richieste. Con l'introduzione della movimentazione autonoma del DDR nello Sprint 2, la stiva deve arricchirsi di una rappresentazione spaziale conforme al sistema di riferimento a coordinate cartesiane richiesto da RobotSmart26.

Separazione tra Topologia Immutabile e Stato Dinamico

La gestione delle informazioni all'interno della stiva presenta due componenti con requisiti di mutabilità opposti:

1. Layout Fisico e Topologia Immutabile: Comprende le coordinate cartesiane (X, Y) delle posizioni operative fisse della stiva, definite in base alla griglia euclidea introdotta nello Sprint 0:
    - HOME: punto di stazionamento iniziale e finale del robot;
    - IOPort: area di carico e accesso antistante la porta;
    - slot5: area per la sosta temporanea e la marcatura barcode;
    - slot1..4: coordinate di deposito dei quattro alloggiamenti definitivi. Questi valori sono costanti di progetto e non cambiano mai nel corso dell'esecuzione. Possono quindi essere letti in totale sicurezza e concorrenza da più attori senza incorrere in condizioni di corsa (race condition).
2. Stato Dinamico di Occupazione: Riguarda la mutabilità dello stato dei singoli alloggiamenti (FREE, RESERVED, OCCUPIED) e l'identificativo dell'eventuale slot riservato correntemente in attesa di deposito. Poiché questo stato è mutevole, deve risiedere esclusivamente all'interno dell'istanza Hold gestita dall'attore orchestratore cargoservice.

Questa distinzione architetturale consente a cargorobot di istanziare una copia locale di sola lettura di Hold, interrogandola unicamente per ricavare le coordinate dei punti di interesse (`getHomeX()/getHomeY()`, `getIOPortX()/getIOPortY()`, `getMarkerX()/getMarkerY()`), senza poter alterare indebitamente lo stato delle riserve della stiva.

Interfaccia IHold

L'entità Hold incapsula la mappa della stiva esponendo metodi sincroni ad alto livello:

Metodo	Chiamante	Scopo e Responsabilità
`reserveNextFreeSlot()`	`cargoservice`	Ricerca sequenziale del primo slot con stato `FREE`. Se trovato, lo marca provvisoriamente come `RESERVED` e ne restituisce l'indice (1..4); se tutti sono impegnati o la stiva è piena, restituisce `NO_SLOT_AVAILABLE` (= -1).
`releaseSlot(slotId)`	`cargoservice`	Chiamato in caso di timeout dei 30s o a seguito di un fallimento del movimento (`transportFailed`): ripristina lo stato dello slot prenotato a `FREE`.
`setSlotOccupied(slotId)`	`cargoservice`	Invocato alla ricezione di `transportDone`: commuta in modo definitivo lo stato dello slot da `RESERVED` a `OCCUPIED`.
`getEntranceX(slotId)` / `getEntranceY(slotId)`	`cargoservice`	Restituisce la coppia di coordinate cartesiane associate all'ingresso dell'alloggiamento target, passate come argomenti nella richiesta `transportContainer`.
`getHoldStateString()`	`cargoservice`	Serializza lo stato corrente dei 4 slot in una stringa compatta (es. `[RESERVED, FREE, FREE, FREE]`) per l'aggiornamento a display tramite l'IOPort.
`isFull()`	`cargoservice`	Controlla se la stiva è piena, ovvero se non c'è nessuno slot in stato `FREE`.

Architettura dei Contesti per lo Sprint 2

Nel nostro Sprint 1 gli attori cargoservice e ioport condividevano lo stesso contesto ctxcargoservice su porta 8120. Nello Sprint 2:

- ctxcargoservice (porta 8120): Ospita il core applicativo cargoservice, l'attore cargorobot e il gateway ioport (connesso alla WebGUI via WebSocket su porta 7070).
- ctxrobotsmart (porta 8020 / host 127.0.0.1): Processo autonomo fornito dal committente che esegue RobotSmart26.

Il modello prodotto

Dall'analisi precedente segue il protocollo dei messaggi del modello qak (sprint2.qak).

```
System sprint2_problem

/*
 * MODELLO DELL'ANALISTA DEL PROBLEMA
 */

// --- Messaggi ereditati dallo Sprint 1 ---
Request loadrequest : loadrequest(IOPORT_STATE)
Reply retrylater    : retrylater(CAUSE, HOLD_STATE) for loadrequest
Reply reject        : reject(HOLD_STATE) for loadrequest
Reply engaged       : engaged(RESERVED_SLOT, HOLD_STATE) for loadrequest

Dispatch button_pushed    : button_pushed(X)
Dispatch containerSensed  : containerSensed(X)
Dispatch updateHoldDisplay : updateHoldDisplay(STATE, HOLD_STATE, MSG)

// --- Nuovi messaggi definiti nell'Analisi dello Sprint 2 ---
Request transportContainer : transportContainer(SLOT, TARGETX, TARGETY)
Reply transportDone        : transportDone(SLOT) for transportContainer
Reply transportFailed      : transportFailed(CAUSE) for transportContainer

// Messaggi di comunincazione definiti da RobotSmart26
Request moverobot          : moverobot(TARGETX, TARGETY, STEPTIME)
Reply moverobotdone        : moverobotok(ARG) for moverobot
Reply moverobotfailed      : moverobotfailed(PLANDONE, PLANTODO) for moverobot

// --- Contesti di Esecuzione ---
Context ctxcargoservice ip [host="localhost" port=8120]
//Context ctxrobotsmart   ip [host="127.0.0.1" port=8020] -> per l'analisi del problema faremo tutto nello stesso contesto

// 1. Mock del servizio esterno RobotSmart26 (per isolare l'analisi del problema)
QActor robotsmart_mock context ctxcargoservice {
	[#
		var Success = 10
	#]
   State s0 initial {
      println("$name | Mock  per Analisi starting...") color green
   }
   Goto idle

   State idle{
      println("$name | waiting...") color green
   }
   Transition t0 whenRequest moverobot -> handleMove

   State handleMove {
      println("$name | moving...") color green
      // Simula il tempo di spostamento del robot sulla griglia
      delay 3000
      // Risponde con successo simulato
      if [#Success > 0#]{
      	[# Success = Success - 1 #]
      	replyTo moverobot with moverobotdone : moverobotok($Success)
      }
      else{
      	[#
      		var Plandone = "none"
      		var Plantodo = "move"
      	#]
      	replyTo moverobot with moverobotfailed : moverobotfailed($Plandone, $Plantodo)
      }
   }
   Goto idle
}


QActor cargorobot context ctxcargoservice {
	[#
	var Slot = "slotX"
	var X = 0
	var Y = 0
	var StepTime = 10
	#]
   State s0 initial {
      println("$name | starting...") color cyan
   }
   Goto idle

   State idle{
      println("$name | waiting for a container...") color cyan
   }
   Transition t0 whenRequest transportContainer -> executeTransportIOPort

   State executeTransportIOPort {
      println("$name | collecting container...") color cyan
      onMsg (transportContainer : transportContainer(SLOT, TARGETX, TARGETY)){
      	[#
      		Slot = payloadArg(0)
      		X = payloadArg(1).toInt()
      		Y = payloadArg(2).toInt()
      		var XIOP = 4
      		var YIOP = 0
      	#]
      	request robotsmart_mock -m moverobot : moverobot ($XIOP, $YIOP, $StepTime)
      }
   }
   Transition t1
      whenReply moverobotdone -> executeTransportSlot5
      whenReply moverobotfailed -> handleFinish

   State executeTransportSlot5{
   	[#
   		var X5 = 2
   		var Y5 = 5
   	#]
      println ("$name | deposit in slot5...") color cyan
      request robotsmart_mock -m moverobot : moverobot ($X5, $Y5, $StepTime)
   }
   Transition t3
      whenReply moverobotdone -> marking
      whenReply moverobotfailed -> handleFinish

   State marking{
      println("$name | marker working...") color cyan
   }
   Transition t2
      whenTime 5000 -> executeTransport

   State executeTransport{
      println ("$name | moving to final slot") color cyan
      request robotsmart_mock -m moverobot : moverobot ($X, $Y, $StepTime)
   }
   Transition t3
      whenReply moverobotdone -> backHome
      whenReply moverobotfailed -> handleFinish

   State backHome{
   	[#
   		var XHOME = 0
   		var YHOME = 0
   	#]
      println ("$name | going back home") color cyan
      request robotsmart_mock -m moverobot : moverobot ($XHOME, $YHOME, $StepTime)
   }
   Transition t4
      whenReply moverobotdone -> handleFinish
      whenReply moverobotfailed -> handleFinish

   State handleFinish{
      onMsg (moverobotdone : moverobotok(ARG)){
      	println ("$name | trasporto finito con successo") color cyan
      	replyTo transportContainer with transportDone : transportDone($Slot)
      }
      onMsg (moverobotfailed : moverobotfailed(PLANDONE, PLANTODO)){
      	[# var Cause = "fail_RobotSmart26" #]
      	println ("$name | trasporto fallito") color cyan
      	replyTo transportContainer with transportFailed : transportFailed($Cause)
      }
   }
   Goto idle
}

QActor cargoservice_ridotto context ctxcargoservice{
	[#
		var Slot = "slot1"
		var X = 2
		var Y = 2
	#]
	State s0 initial {
		println("")
	}
	Goto handleContainer

	State handleContainer {
	println("$name | Container rilevato -> richiesta di movimento del robot") color magenta
	request cargorobot -m transportContainer : transportContainer ($Slot, $X, $Y)
   }
   Transition t0
      whenReply transportDone -> handleTransportComplited
      whenReply transportFailed -> handleTransportFailed

   State handleTransportComplited{
      // aggiornamento display e liberazione area sensore
      println("$name | il container è stato prelevato, marcato e depositato nello slot assegnato") color magenta
   }
   Transition t1
      whenTime 5000 -> handleContainer

   State handleTransportFailed{
      // aggiornamento dispay e gestione dell'errore
      println("$name | si è verificato un errore durante il trasporto. OUT OF SERVICE!") color magenta
   }
}
```

Test plans

I casi di test mirano a validare l'orchestrazione della missione di trasporto e l'integrazione con il servizio esterno:

Caso di Test	Precondizione	Input / Azione	Risultato Atteso
Missione Completa	Robot in HOME, stiva non piena, sensore IOPort validato.	`cargoservice` invia `transportContainer(SLOT, X, Y)` a `cargorobot`.	`cargorobot` emette in sequenza le `moverobot` verso IOPort, slot5, slot riservato e HOME. Termina con reply `transportDone`.
Sosta di Marcatura (Slot 5)	Robot giunto alle coordinate dello slot5.	Emissione `moverobotok` da `RobotSmart26`.	Il robot permane nello stato `atSlot5` per la durata prefissata della marcatura prima di dirigersi allo slot finale.
Gestione Guasto Robot	Robot in movimento lungo una tratta.	`RobotSmart26` risponde con `moverobotfailed`.	`cargorobot` interrompe il piano, transita nello stato `halted` e invia `transportFailed` a `cargoservice`.
Aggiornamento Stiva al Rientro	Missione terminata con successo (`transportDone`).	Ricezione esito positivo da parte di `cargoservice`.	Lo slot riservato commuta a OCCUPIED, il sistema torna in idle e il display mostra lo stato aggiornato.

Project

L'attore cargorobot

L'attore implementa la seguente sequenza:

1. Movimento da HOME a IOPort;
2. Movimento da IOPort a slot5 (con sosta di marcatura barcode);
3. Movimento da slot5 allo slot finale riservato;
4. Rientro in HOME e chiusura della missione.

```
// ---------------- Attore CARGOROBOT con recupero in caso di guasto ----------------------------------------
QActor cargorobot context ctxcargoservice
    withobj h using "cargoservice.Hold()"{
   [#
      var Slot     = 0
      var X        = 0      // coordinate dello slot finale
      var Y        = 0
      var StepTime = 345

      val MaxHomeAttempts = 3          // tentativi massimi di rientro in HOME

      var Failed       = false         // true se una tappa fallisce
      var FailCause    = "fail_RobotSmart26"
      var HomeAttempts = 0
      var NearHome     = false         // true se manca solo l'ultimo passo per casa
   #]

   State s0 initial {
      [#
            var XHOME = h.getHomeX()
            var YHOME = h.getHomeY()
      #]
      println("$name | starting: mind set at HOME (robot assumed physically at HOME, facing down)") color blue
      forward robotsmart -m setplanbuildelay : value(0)
      forward robotsmart -m setrobotstate : setpos($XHOME,$YHOME,down)
      delay 1000
   }
   Goto idle

   State idle {
      println("$name | waiting for a container...") color blue
   }
   Transition t0
      whenRequest transportContainer -> extractRequest

   State extractRequest {
      onMsg (transportContainer : transportContainer(SLOT, TARGETX, TARGETY)) {
         [#
            Slot         = payloadArg(0).toInt()
            X            = payloadArg(1).toInt()
            Y            = payloadArg(2).toInt()
            Failed       = false
            FailCause    = "fail_RobotSmart26"
            HomeAttempts = 0
         #]
      }
   }
   Goto moveIOPort

   // --- 1. HOME -> IOPort ---
   State moveIOPort {
      [#
            var XIOP = h.getIOPortX()
            var YIOP = h.getIOPortY()
      #]
      println("$name | moving to IOPort ($XIOP, $YIOP)") color blue
      request robotsmart -m moverobot : moverobot($XIOP, $YIOP, $StepTime)
   }
   Transition t1
      whenTime 30000            -> moveTimeout
      whenReply moverobotdone   -> collectContainer
      whenReply moverobotfailed -> legFailed

   State collectContainer {
      println("$name | collecting container...") color blue
      delay 2000
   }
   Goto moveSlot5

   // --- 2. IOPort -> slot5 ---
   State moveSlot5 {
      [#
            var X5 = h.getMarkerX()
            var Y5 = h.getMarkerY()
      #]
      println("$name | moving to slot5 ($X5, $Y5)") color blue
      request robotsmart -m moverobot : moverobot($X5, $Y5, $StepTime)
   }
   Transition t2
      whenTime 30000            -> moveTimeout
      whenReply moverobotdone   -> marking
      whenReply moverobotfailed -> legFailed

   State marking {
      println("$name | marker working...") color blue
   }
   Transition t3
      whenTime 5000 -> moveFinalSlot

   // --- 3. slot5 -> slot finale ---
   State moveFinalSlot {
      println("$name | moving to final slot ($X, $Y)") color blue
      request robotsmart -m moverobot : moverobot($X, $Y, $StepTime)
   }
   Transition t4
      whenTime 30000            -> moveTimeout
      whenReply moverobotdone   -> depositContainer
      whenReply moverobotfailed -> legFailed

   State depositContainer {
      println("$name | depositing container...") color blue
      delay 2000
   }
   Goto startHoming

   // --- Gestione fallimento di una tappa ---
   State legFailed {
      onMsg (moverobotfailed : moverobotfailed(PLANDONE, PLANTODO)) {
         println("$name | tappa fallita! PLANDONE=${payloadArg(0)} PLANTODO=${payloadArg(1)}") color red
      }
      [# Failed = true #]
   }
   Goto startHoming

   State moveTimeout {
      println("$name | nessuna risposta da robotsmart entro 30s") color red
      [# Failed = true #]
   }
   Goto finishKo

   // --- 4. Rientro in HOME ---
   State startHoming {
      [# HomeAttempts = 0 #]
   }
   Goto goHome

   State goHome {
      [#
            HomeAttempts++
            var XHOME = h.getHomeX()
            var YHOME = h.getHomeY()
      #]
      println("$name | going back home (tentativo $HomeAttempts di $MaxHomeAttempts)") color blue
      request robotsmart -m moverobot : moverobot($XHOME, $YHOME, $StepTime)
   }
   Transition t5
      whenTime 30000            -> moveTimeout
      whenReply moverobotdone   -> tuneHome
      whenReply moverobotfailed -> homeMoveFailed

   State homeMoveFailed {
      onMsg (moverobotfailed : moverobotfailed(PLANDONE, PLANTODO)) {
         [#
            val Todo = payloadArg(1).filter { c -> c.isLetter() }
            NearHome = (Todo == "w" || Todo == "")
         #]
         println("$name | rientro in HOME fallito: PLANDONE=${payloadArg(0)} PLANTODO=${payloadArg(1)} -> quasi a casa: $NearHome") color red
      }
   }
   Goto tuneHome if [# NearHome #] else retryHome

   State retryHome {
      println("$name | lontano da HOME, ritento il rientro") color magenta
   }
   Goto goHome if [# HomeAttempts < MaxHomeAttempts #] else homeGiveUp

   State homeGiveUp {
      println("$name | impossibile rientrare in HOME dopo $MaxHomeAttempts tentativi") color red
      [# Failed = true #]
   }
   Goto finishKo

   // --- Allineamento in HOME ---
   State tuneHome {
      println("$name | tuning at home...") color blue
      request robotsmart -m tuneAtHome : tuneAtHome(end)
   }
   Transition t6
      whenTime 20000     -> tuneTimeout
      whenReply tuneDone -> endHoming

   State tuneTimeout {
      println("$name | tune NON terminato entro 20s") color red
      [# Failed = true #]
   }
   Goto finishKo

   State endHoming {
      println("$name | tune completato, robot riallineato in HOME") color blue
      forward robotsmart -m setrobotstate : setpos(0,0,down)
      delay 300
   }
   Goto finishKo if [# Failed #] else finishOk

   State finishOk {
      println("$name | trasporto finito con successo") color blue
      replyTo transportContainer with transportDone : transportDone($Slot)
   }
   Goto idle

   State finishKo {
      println("$name | trasporto fallito ($FailCause)") color red
      replyTo transportContainer with transportFailed : transportFailed($FailCause)
   }
   Goto idle
}
```

cargoservice

cargoservice delega il movimento a cargorobot dopo l'accettazione e il rilevamento del container:

```
   // Stato modificato
   State handleContainer {
      println("$name | Container rilevato -> movimento robot") color magenta
      [#
         Transporting = true
         var TargetX = h.getEntranceX(CurrentReservedSlot)
         var TargetY = h.getEntranceY(CurrentReservedSlot)
      #]
      request cargorobot -m transportContainer : transportContainer($CurrentReservedSlot, $TargetX, $TargetY)
   }
   Goto waitTransport

   // Attesa della reply del robot
   State waitTransport {
      println("$name | In attesa della fine del trasporto...") color magenta
   }
   Transition t2
      whenReply transportDone   -> endTransport
      whenReply transportFailed -> handleOutOfService
      whenRequest loadrequest   -> handleBusyRequest
      whenMsg outOfService      -> handleOutOfService

   State endTransport {
      println("$name | Container portato nello slot definito") color magenta
      [#
         h.setSlotOccupied(CurrentReservedSlot)
         CurrentReservedSlot = -1
         var CurrentHoldState = h.getHoldStateString()
         var StateStr = "'LOADED'"
         var LoadedMsgStr = "'Container preso in carico'"
      #]
      forward ioport -m updateHoldDisplay : updateHoldDisplay($StateStr, $CurrentHoldState, $LoadedMsgStr)
      delay 1500
   }
   Goto idle

   // --- Out of service: guasto sensore o guasto robot ---
   State handleOutOfService {
      println("$name | GUASTO RILEVATO: Sistema in Out Of Service") color magenta
      onMsg(outOfService : outOfService(CAUSE)) {
         [# OosMsg = "'Guasto sensore'" #]
      }
      onMsg(transportFailed : transportFailed(CAUSE)) {
         [# OosMsg = "'Guasto robot'" #]
      }
      [#
         IsOutOfService = true

         // Lo slot viene rilasciato solo se il robot NON sta trasportando
         // (altrimenti il container potrebbe essere gia' nel robot)
         if (!Transporting && CurrentReservedSlot != -1) {
            h.releaseSlot(CurrentReservedSlot)
            CurrentReservedSlot = -1
         }

         var CurrentHoldState = h.getHoldStateString()
         var StateStr = "'OUT_OF_SERVICE'"
         var LedRed = "'red'"
      #]
      forward ioport -m led : led($LedRed)
      forward ioport -m updateWorkingState : updateWorkingState($StateStr)
      forward ioport -m updateHoldDisplay : updateHoldDisplay($StateStr, $CurrentHoldState, $OosMsg)
   }
   Goto outOfServiceState

   // Risposta retrylater mentre il sistema e' engaged oppure sta trasportando.
   // Torna nello stato di attesa giusto in base al flag Transporting.
   State handleBusyRequest {
      onMsg(loadrequest : loadrequest(IOPORT_STATE)) {
         [#
            var CurrentHoldState = h.getHoldStateString()
            var SystemEngagedStr = "'system_engaged'"
         #]
         replyTo loadrequest with retrylater : retrylater($SystemEngagedStr, $CurrentHoldState)
         println("$name | retrylater($SystemEngagedStr, $CurrentHoldState)") color magenta
      }
   }
   Goto waitTransport if [# Transporting #] else engagedWaitContainer
```

Implementazione della Classe POJO Hold.java

Si implementa l'interfaccia presentata nell'analisi del problema:

```
package cargoservice;

import cargoservice.IHold.SlotState;

public class Hold implements IHold{

   private final SlotState[] mainSlots = new SlotState[NUM_MAIN_SLOTS];
   /*
    * INGRESSI SCELTI:
    * - slot1 = (0,2)
    * - slot2 = (0,3)
    * - slot3 = (2,2)
    * - slot4 = (2,3)
    */
   private final int[] slotEntranceX = {0,0,2,2};
   private final int[] slotEntranceY = {2,3,2,3};
   private final int[] homePos = {0,0};
   private final int[] ioportPos = {4,0};
   private final int[] markerPos = {2,5};
```

```
    private static final int X = 0;
    private static final int Y = 1;

    public Hold() {
        for (int i = 0; i < NUM_MAIN_SLOTS; i++) {
            mainSlots[i] = SlotState.FREE;
        }
    }

    @Override
    public synchronized int reserveNextFreeSlot() {
        for (int i = 0; i < NUM_MAIN_SLOTS; i++) {
            if (mainSlots[i] == SlotState.FREE) {
                mainSlots[i] = SlotState.RESERVED;
                return i + 1;
            }
        }
        return NO_SLOT_AVAILABLE;
    }

    @Override
    public synchronized void setSlotOccupied(int slotNumber) {
        if (isValidSlot(slotNumber)) {
            mainSlots[slotNumber - 1] = SlotState.OCCUPIED;
        }
    }

    @Override
    public synchronized void releaseSlot(int slotNumber) {
        if (isValidSlot(slotNumber)) {
            mainSlots[slotNumber - 1] = SlotState.FREE;
        }
    }

    @Override
    public synchronized boolean isFull() {
        for (int i = 0; i < NUM_MAIN_SLOTS; i++) {
            if (mainSlots[i] == SlotState.FREE) {
                return false;
            }
        }
        return true;
    }

    private boolean isValidSlot(int slotNumber) {
        return slotNumber >= 1 && slotNumber <= NUM_MAIN_SLOTS;
    }

    @Override
    public synchronized int getEntranceX(int slotNumber) {
        if (isValidSlot(slotNumber)) {
                return slotEntranceX[slotNumber-1];
        }
        return -1;
    }

    @Override
    public synchronized int getEntranceY(int slotNumber) {
        if (isValidSlot(slotNumber)) {
                return slotEntranceY[slotNumber-1];
        }
        return -1;
    }

    @Override
    public synchronized String getHoldStateString() {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < NUM_MAIN_SLOTS; i++) {
            sb.append(mainSlots[i]);
            if (i < NUM_MAIN_SLOTS - 1) {
                sb.append(", ");
            }
        }
        sb.append("]");
        return sb.toString(); // Restituisce es: "[RESERVED, FREE, FREE, FREE]"
    }

    @Override
    public SlotState getSlotState(int slotNumber) {
        if (isValidSlot(slotNumber)) {
                return mainSlots[slotNumber-1];
        }
        return null;
    }

        @Override
        public synchronized int getHomeX() {
                return homePos[X];
        }

        @Override
        public synchronized int getHomeY() {
                return homePos[Y];
        }

        @Override
        public synchronized int getIOPortX() {
                return ioportPos[X];
        }

        @Override
        public synchronized int getIOPortY() {
                return ioportPos[Y];
        }

        @Override
        public synchronized int getMarkerX() {
                return markerPos[X];
        }

        @Override
        public synchronized int getMarkerY() {
                return markerPos[Y];
        }
}
```

Testing

Deployment

Il deployment dello Sprint 2 richiede l'esecuzione concorrente del simulatore del robot e dell'applicazione QAk

Porta	Nodo / Processo	Ruolo
8120	`ctxcargoservice`	Canale TCP QAk per gli attori dell'applicazione.
8020	`ctxrobotsmart`	Canale TCP del servizio RobotSmart26 (interrogato da `cargorobot`).
7070	`IOPortWsAdapter`	Web server Javalin (GUI client su `/ioport_gui.html` e WebSocket).

Definizione dello Sprint3

Requisiti Selezionati per lo Sprint 3

- Integrazione del Sonar Fisico/Simulato: Realizzare l'attore sonar collegato a un dispositivo hardware reale (Raspberry PicoW con sensore a ultrasuoni) o a un emulatore fedele per rilevare la distanza D.
- Gestione degli Stati di Anomalia Sonar: Validare il carico solo se D < DFREE/2 per almeno 3 secondi; commutare il sistema nello stato Out of service qualora persista una lettura D > DFREE per oltre 3 secondi.
- Attuatore LED: Implementare il controllo dell'attuatore fisico (o simulato su Raspberry PicoW) per il lampeggio in engaged e la luce fissa/rossa in Out of service.

Goal dello Sprint 3

Progettare e realizzare il sottosistema telemetrico sonar e l'attuatore led, integrando il firmware su microcontrollore o nodo dedicato. Completare la gestione delle anomalie telemetriche con transizione nello stato Out of service a display e gestire la segnalazione luminosa nelle diverse fasi operative.

Maintenance

Team di Sviluppo

💻 GitHub Project Repository

Arianna Buffoni
arianna.buffoni@studio.unibo.it

Desirée Pellegrini
desiree.pellegrini@studio.unibo.it

Gianluca Varisco
gianluca.varisco@studio.unibo.it